

득실

// 예금 깰까, 돈 빌릴까?

목돈이 필요한 순간의 두 선택지를 **같은 시점 · 같은 세후 기준으로 환산**해 나란히 보여주는 손익 비교 서비스

기간 2026.07 - 08

인원 6명

저장소 github.com/Jys122211/PJT-29-2-

시연 [시연 영상 URL](#)

01 ABOUT

왜 만들었나

사용자가 마주하는 상황

갑작스럽게 목돈이 필요해진 상황. 만기 7개월 남은 예금을 깨면 쌓아둔 이자를 대부분 잃고, 대출을 받으면 이자를 새로 냅니다.

어느 쪽이 나은지 알려면 **중도해지이율 구간 표**, 남은 기간, 신용등급별 대출금리, 중도상환 수수료를 전부 알아야 합니다. 금융 상품 구조에 익숙하지 않으면 비교의 출발점조차 잡기 어렵습니다. 결국 감으로 결정하거나 창구에 묻게 됩니다.

서비스가 하는 일

보유 예금과 필요 금액만 입력하면, 두 선택지의 총비용을 **같은 시점 · 같은 세후 기준으로 환산**해 비교합니다.

해지가 유리하면 해지하라고 말합니다. 상품을 파는 계산기가 아니라, 사용자가 스스로 판단할 근거를 주는 것을 목표로 했습니다.

02 FEATURES

주요 기능

예금을 등록하고 대출 조건을 좁혀 나가면 마지막에 두 선택지의 손익이 나란히 나옵니다.

예금 등록 → 필요 금액 → 대출 종류 → 자격조건 → 우대금리 → 결과 비교

01 예금 등록 — 캡처와 직접 입력

은행 앱 화면을 올리면 OCR이 은행명·계좌번호·금리·가입금액·만기일을 읽어 폼을 채웁니다. **인식하지 못한 칸은 비워 두고** 사용자가 이어서 채웁니다. 캡처가 없으면 처음부터 직접 입력해도 됩니다.

어느 경로로 들어왔든 **같은 검증이 걸립니다**. 계좌번호 자릿수, 존재하지 않는 날짜, 가입일이 오늘 이후인 경우, 만기일이 오늘 이전인 경우를 각각 잡아 해당 칸에 빨간 테두리와 문구로 알립니다. OCR이 채운 값도 예외가 아니어서, 잘못 읽은 값은 등록 전에 드러납니다.

금액과 금리는 입력하는 동안 자릿수 구분과 소수점 자릿수를 맞춰 주고, 그 과정에서 **커서 위치가 흐트러지지 않도록** 처리했습니다. 등록 화면과 수정 화면이 같은 규칙을 공유합니다.

02 단계별 자격·우대조건 확인

질문을 한 번에 쓰지 않고 답한 만큼만 다음 질문을 노출합니다. 자격조건을 통과한 상품만 후보로 남기고, 해당하는 우대금리를 합산합니다. 새 질문이 화면 밖에 생기면 자동으로 스크롤해 시선을 옮겨 줍니다.

03 손익 비교 계산

신용대출과 전세대출을 각각 다른 규칙으로 계산합니다. 부분해지 가능 여부, 만기 목돈 상환 여부에 따라 네 가지 경우로 나눕니다.

04 득실기록

결과는 비교 시점의 상품명·금리를 스냅샷으로 저장합니다. 상품 정보가 나중에 바뀌어도 과거 기록이 흔들리지 않습니다.

03 TEAM

팀 구성과 역할

6명 전원이 데이터베이스 설계에 참여했습니다. 파트를 프론트·백엔드로 나누지 않고, 각자 맡은 도메인을 화면부터 API·쿼리·테이블까지 한 사람이 끝까지 책임지는 방식으로 진행했습니다.

조윤상 신용대출 Core 프로세스 설계 및 구현 Full-stack Database 29회차	신윤지 특실기록 · 홈 구현, 대출 데이터 정제 Full-stack Database 30회차
안상우 손익 계산 로직 설계·구현 및 결과 시각화 Backend Database 29회차	이지은 자산관리 API 구축 및 OCR 연동 Full-stack Database 30회차
정성윤 회원관리, 회원 개인금융정보 검증 및 저장 Full-stack Database 29회차	조용진 전세대출 구현, 계산 로직 설계·구조 개선 Full-stack Database 30회차

04 SKILLS

기술 스택

Spring MVC · MyBatis · JWT는 과정에서 지정된 스택입니다. 아래 표에는 팀이 직접 판단해 고른 것만 적었습니다.

FRONTEND Vue 3.5 Composition API Pinia 3 Vue Router 5 Vite 7 Axios
BACKEND Spring MVC 5.3 Java 17 MyBatis 3.4 Spring Security 5.8 JWT · jjwt 0.11 Gradle Swagger 2
DATABASE MySQL 8 InnoDB utf8mb4 AES-256-GCM BigDecimal
수 집 Java · Jsoup 금융상품통합비교공시 API SHA-256 스냅샷 OCR

선택	대안	고른 이유
BigDecimal	double	금액 계산에서 부동소수점 오차가 누적되면 결과를 재현할 수 없음
계좌번호 암호화	평문 그대로 저장	그대로 저장하면 DB에 접속할 수 있는 사람은 누구나 모든 계좌번호를 볼 수 있음 . 암호화 키를 DB 밖(환경변수)에 두어 DB만으로는 읽지 못하게 함
스냅샷 저장	상품 테이블 참조	비교 시점 값을 복사해 상품이 바뀌어도 과거 이력이 흔들리지 않도록
탈퇴 시 이메일 익명화	물리 삭제 · 그대로 보존	물리 삭제는 비교 이력까지 지우고, 그대로 두면 재가입이 막힘. 익명화가 둘을 동시에 해결

05 PROBLEMS SOLVED

문제를 정의하고, 구조를 설계하고, 결과를 만든 경험들

계산이 성립하지 않던 문제부터, 데이터가 조용히 사라진 사고를 추적한 일까지 네 가지입니다.

01

안상우

조용진

금리와 상환기간이 서로를 필요로 해 계산이 성립하지 않았다

PROBLEM

대출금리는 금리변동주기(3·6·12개월)마다 다릅니다. 어느 주기를 적용할지 정하려면 **실제 상환이 몇 개월 걸리는지**를 알아야 하고, 상환 개월을 구하려면 **금리가 먼저 필요**했습니다. 잘못 고르면 사용자가 실제보다 비싼 금리로 계산된 결과를 보게 되는 문제였습니다.

APPROACH

두 방식을 비교했습니다. ① 예금 잔여기간으로 주기를 **추정**, ② 가장 짧은 주기부터 **시험 대입**해 수렴. ①은 조기 완납 시 과대추정된 구간에 갇혀 실제보다 높은 금리가 선택되어 버렸습니다.

```
int ratePeriod = 3; // 가장 유리한 주기부터
while (true) {
    var r = simulate(amount, rates.get(ratePeriod), monthlyPayment);
    if (r.repaymentMonths() <= ratePeriod || ratePeriod == 12) {
        return resolve(ratePeriod, r); // 금리·상환개월 동시 확정
    }
    ratePeriod = (ratePeriod == 3) ? 6 : 12;
}
```

선택 이유 — 주기가 짧을수록 금리가 낮으므로, 3개월부터 올라가면 조건을 만족하는 첫 구간이 곧 사용자에게 가장 유리한 금리가 됩니다.

BigDecimal

PMT

시험 대입

RESULT

3·6·12개월 전 구간에서 상환 흐름이 검증되고, 조기 완납 시 발생하던 과대 금리 선택이 제거됐습니다.

계산 불가 → 후보 3개 중 자동 선택

측정 상환 전략(만기 목돈 상환 O/X) × 금리주기 3구간 교차 검증 단위 테스트로 기대 금리·상환개월 고정. 손계산 명세서 4건과 대조해 전부 일치.

LESSON LEARNED

순환 참조를 방정식으로 풀려다 막혔습니다. **은행이 주는 선택지가 셋뿐**이라는 도메인 사실을 확인한 순간, 무한한 문제가 후보 세 개짜리 탐색으로 바뀌었습니다. 기술로 풀기 전에 도메인을 먼저 봐야 했습니다.

02 안상우

반올림 시점에 따라 결과가 달라져 계산을 신뢰할 수 없었다

PROBLEM

금리 계산은 나눗셈이 반복됩니다. 경과월수 ÷ 계약월수처럼 딱 떨어지지 않는 값이 많아 **어디서 반올림하느냐에 따라 최종 금액이 달라졌습니다**. 금융 서비스에서 1원이 어긋나면 계산 전체의 신뢰가 무너진다고 판단했습니다.

APPROACH

`double`은 부동소수점 오차가 누적되어 배제하고 **BigDecimal**로 전 구간을 계산했습니다. 중간 정밀도를 20자리로 유지하고, 원 단위 반올림은 **최종 결과에서 한 번만** 수행하도록 규칙을 통일했습니다.

수수료도 항목별로 반올림해 더하지 않고 소수 그대로 합산한 뒤 마지막에 처리합니다.

선택 이유 — 단계마다 반올림하면 코드는 단순해지지만 결과를 재현할 수 없습니다. 재현 가능성을 우선해 정밀도를 끝까지 들고 가는 쪽을 택했습니다.

BigDecimal

CALC_SCALE = 20

JUnit

RESULT

같은 입력에 항상 같은 결과가 나오고, 손계산 명세서와 원 단위까지 일치합니다.

단계마다 반올림 → 최종 1회

측정 계산기 클래스별 단위 테스트로 시나리오마다 기대값 고정. 전체 58건 통과 · 실패 0.

LESSON LEARNED

같은 공식이어도 반올림 시점, 개월 수를 세는 기준, 세금을 빼는 순서에 따라 결과가 달라집니다. **규칙을 정하고 팀 전체가 같은 기준으로 계산하도록 맞추는** 일이, 공식을 코드로 옮기는 일보다 오래 걸렸습니다.

자동 수집은 반드시 재실행되고, 반드시 실패한다

PROBLEM

금리는 매일 바뀌므로 상품 정보 수집을 자동화해야 했습니다. 그런데 스케줄러로 돌리면 같은 배치가 여러 번 실행되고, 은행 페이지 구조가 바뀌면 파싱이 깨집니다. 그때마다 상품 테이블에 중복이 쌓이거나 값이 반쯤 덮여 쓰이면, 그 위에서 계산한 손익은 근거가 없어집니다.

멘토 피드백에서도 같은 지점을 지적받았습니다. “실패했을 때 어떻게 되는지, 두 번 실행했을 때 어떻게 되는지”

APPROACH

수집을 한 번의 실행 단위로 보고, 그 단위를 테이블로 만든 뒤 세 겹의 안전장치를 얹었습니다.

- **배치 자체를 기록** — 상태 5단계(RUNNING / SUCCESS / PARTIAL / FAILED / REJECTED), 대상·성공·실패 건수, 라이브 반영 여부를 남깁니다.
- **변경이 없으면 쓰지 않는다** — 수집 결과를 SHA-256으로 요약해 직전과 같으면 적재를 건너뛵니다.
- **테이블 성격에 맞는 층에서 막는다** — 덮어쓰는 라이브는 애플리케이션이 UPSERT로, 쌓기만 하는 이력은 DB의 복합 UNIQUE로 막습니다.

라이브 UPDATE (상품, 기간, 등급) → 0행이면 INSERT
2행 이상 걸리면 이미 중복 → 예외로 배치 롤백

이력 INSERT만 하므로 감지할 기회가 없음
UNIQUE (products_history_id, rate_period_months, credit_grade)

Jsoup

SHA-256

UPSERT

복합 UNIQUE

RESULT

배치를 49회 실행했고 라이브·이력 어디에도 중복이 생기지 않았습니다. 실패한 2회는 트랜잭션이 롤백돼 라이브 테이블에 흔적을 남기지 않았습니다.

~~49회 실행~~ → 13회 적재 · 36회 스킵

중복 0건

측정 운영 DB에서 라이브·이력 전 테이블의 전체 행 수와 고유 조합 수를 대조. 배치 이력에서 성공 45 / 부분 2 / 실패 2 확인.

LESSON LEARNED

중복 방지를 코드로만 막고 있던 구간이 있었습니다. 크롤러를 통하지 않는 경로에서는 아무것도 막히지 않는다는 걸 뒤늦게 확인했습니다. 애플리케이션이 지키는 규칙과 DB가 강제하는 규칙을 구분해 적는 습관이 생겼습니다.

계좌번호를 암호화했더니, 이번엔 값이 조용히 사라졌다

PROBLEM

계좌번호를 평문으로 저장하면 DB에 접속할 수 있는 사람은 누구나 전부 볼 수 있습니다. 운영자가 사용자의 계좌번호를 볼 수 있어서는 안 된다는 멘토 지적을 받았습니다.

그런데 암호화를 켜 뒤 **일부 예금의 계좌번호가 화면에 비어 보였습니다**. 오류도, 예외 로그도 없었습니다. 저장은 성공한 것처럼 보이는데 값만 없는 상태여서 원인을 좁히기 어려웠습니다.

APPROACH

먼저 **AES-256-GCM**으로 암호화해 저장하고, 키는 소스가 아닌 **환경변수**에 두어 DB 접근만으로는 복호화할 수 없게 했습니다. 매 호출마다 새 IV를 생성해 같은 계좌번호도 저장값이 달라집니다.

저장 형식 : Base64(IV(12B) + 암호문 + 인증태그(16B))

암호화 실패 → 예외 (평문이 저장되는 사고 차단)

복호화 실패 → null 반환 (마이그레이션 전 데이터가 있어도 화면 유지)

키 없이 기동 → 즉시 중단 (조용히 평문 저장되는 것 방지)

선택 이유 — CBC 대신 GCM을 쓴 것은 암호화와 위변조 검증을 함께 하기 위해서입니다. 누군가 DB의 암호문을 고치면 복호화 단계에서 인증 태그 검증에 실패합니다.

사라진 값은 **DB에 실제로 들어 있는 길이부터 재서** 찾았습니다. 정상 암호문이 56자인데 컬럼은 **VARCHAR(30)**이었습니다. 평문 계좌번호 14자리를 기준으로 잡은 길이였습니다.

평문 32841098320412 14자
암호문 IV(12B) + 암호문 + 태그(16B) → Base64 → 56자
컬럼 VARCHAR(30) ← 26자 부족

암호화 이전에 등록된 **평문 1건**도 섞여 있었습니다. 이 값은 Base64로 해석하면 10바이트라 IV 길이(12바이트)에 못 미쳐 복호화가 실패합니다. 실패 시 **null**을 돌려주는 설계 덕분에 화면은 깨지지 않았지만, 그 때문에 **문제가 더 늦게 드러났습니다**.

판단 — 유실된 행을 지우려 했으나 비교 이력이 외래키로 참조하고 있었습니다(**Error 1451**). 이력은 서비스의 핵심 산출물이라, **지우지 않고 컬럼만 넓히는 쪽**을 택했습니다.

AES-256-GCM

환경변수 키

VARCHAR(255)

FK 제약

RESULT

DB를 직접 조회해도 계좌번호는 Base64 문자열로만 보이고, 같은 계좌를 두 번 저장해도 저장값이 달라 암호문 비교로 동일 계좌를 추정할 수 없습니다.

~~VARCHAR(30)~~ → 저장 실패 → **VARCHAR(255)** · 정상

측정 전체 예금 행의 계좌번호 길이 분포를 조회해 56자 암호문 / 평문 / NULL 건수를 구분. 동일 계좌 2회 저장 시 서로 다른 암호문 생성 확인. 환경변수 미설정 시 기동이 중단되는지 확인.

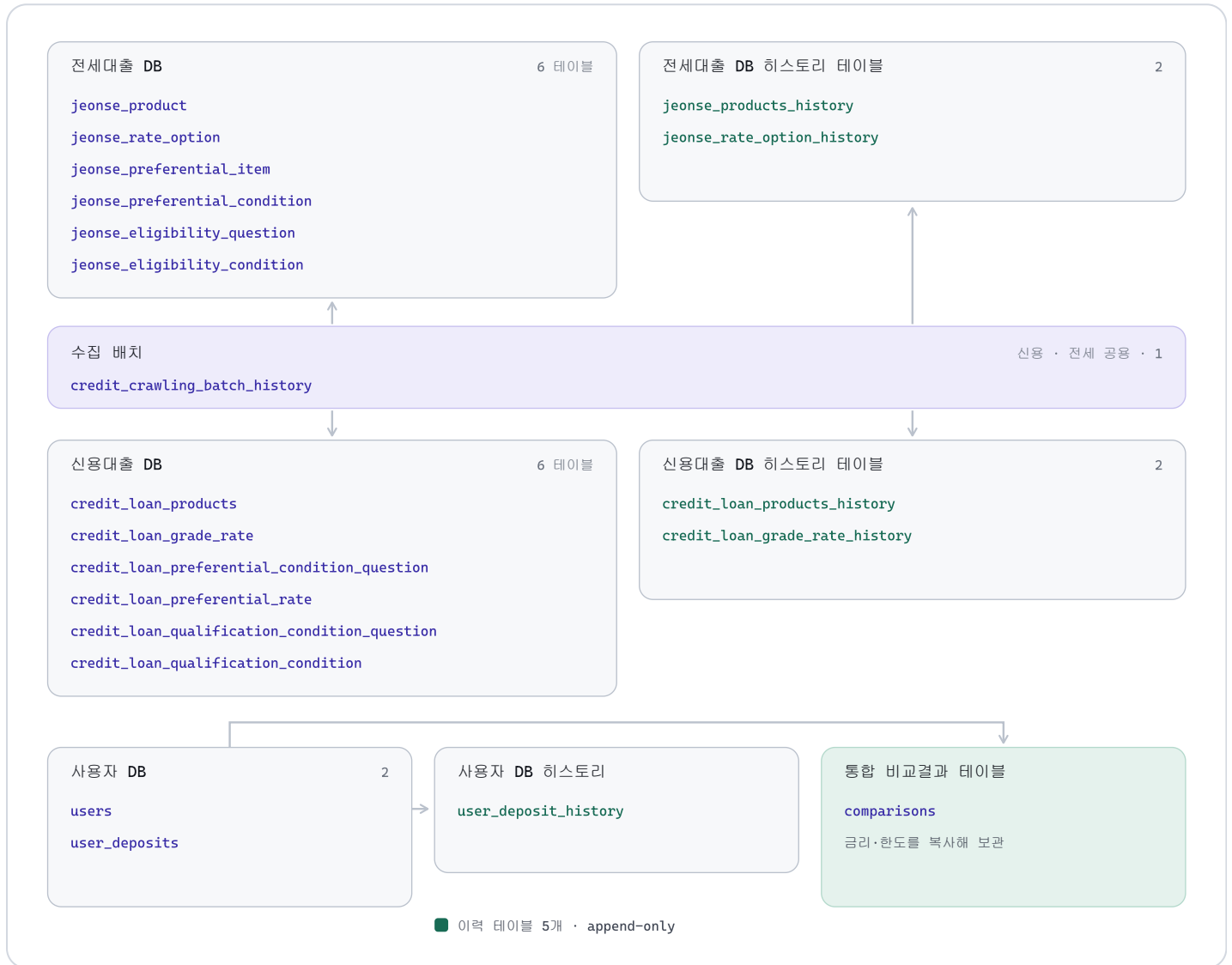
LESSON LEARNED

평문 14자리를 보고 `VARCHAR(30)` 이면 넉넉하다고 판단했는데, 실제 저장되는 값은 56자였습니다. **변환을 거치는 값은 변환 후 크기를 먼저 재야 합니다.** 그리고 실패를 `null`로 삼키는 설계는 화면을 지켜 주지만 문제를 늦게 드러냅니다. 삼킬 곳과 터뜨릴 곳을 나눠야 했습니다.

06 DATA MODEL

데이터 모델

사용자가 직접 입력하는 예금과, 크롤링으로 채우는 대출 상품이 비교 결과 테이블에서 만납니다. 성격이 다른 두 갈래를 하나의 규칙으로 묶는 것이 설계의 축이었습니다.



도메인	테이블	설계 포인트
사용자	3	계좌번호 컬럼 암호화 · 탈퇴 시 이메일 익명화 · 예금 변경 이력 append-only
신용대출	8	상품 · 등급별 금리 · 우대 · 자격 + 이력 2
전세대출	8	상품 · 금리옵션 · 우대 · 자격 + 이력 2
수집 · 결과	2	배치 이력(신용·전세 공용) · 비교 결과 스냅샷

신용대출 물리 모델

라이브 6 · 이력 2 · 공용 배치

[문서 보기 →](#)

(<https://claude.ai/code/artifact/401ff785-de0f-4d38-b973-109f200a29ec>)

전세대출 물리 모델

라이브 6 · 이력 2 · 공용 배치

[문서 보기 →](#)

(<https://claude.ai/code/artifact/325d2fcc-eed8-4fd3-83cc-71ca3897dadf>)

07 FEEDBACK

피드백과 개선

멘토 네 분께 받은 피드백 중 서비스 구조를 바꾼 것만 추렸습니다. 아직 반영하지 못한 지적은 **08 남은 과제**에 모았습니다.

멘토	지적	대응
전태경	계좌번호가 평문으로 저장된다	AES-256-GCM 암호화. 키는 환경변수에 두어 DB 접근만으로는 복호화할 수 없게 함
전태경	크롤링이 실패했을 때·두 번 실행했을 때 어떻게 되는지	배치 이력 테이블 + 스냅샷 해시 + 복합 UNIQUE 3중 방어. 49회 실행에 중복 0건
전태경	운영자가 금리 변경 시점을 알 수 없고, 계산 결과도 남지 않는다	이력 테이블 5개 신설, 비교 결과는 계산 시점의 금리·한도를 복사해 보관
전태경	자바에서 double 대신 DECIMAL을 쓰라	BigDecimal 전면 적용. 중간 정밀도 20자리, 반올림은 최종 1회
지석호	예금삭제·로그아웃 alert이 화면마다 다르다	Composable + 공통 모달로 통일
조용훈	API 명세서를 써서 프론트·백엔드가 합의하라	엔드포인트 29개 명세 작성 후 양쪽이 같은 문서를 참조

08 OPEN ISSUES

남은 과제

발표 시점 기준으로 정리하지 못한 것들입니다. 감추기보다 적어 두는 편이 다음 단계를 정하는 데 낫다고 봤습니다.

- **이력 조회 화면 없음** — 저장만 동작하지만 API와 화면이 없어 DB 직접 조회로만 확인합니다. 의도한 범위 밖입니다.
- **OCR 동기 처리** — 인식이 5~10초 걸리는 동안 토크트 스레드를 점유합니다. 처리 ID를 먼저 발급하고 폴링으로 조회하는 방식으로 바뀌어 합니다. [전태경 멘토](#)
- **상품 코드 체계 도입** — 현업은 대·중·소분류 코드로 상품을 나눕니다. 상품명도 별도 테이블로 분리해야 합니다. [지석호 멘토](#)

